

GPU SETUP

Nabila Puspita Firdi, S.T., M.Sc.

Teknik Biomedik

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

1





CPU AND GPU

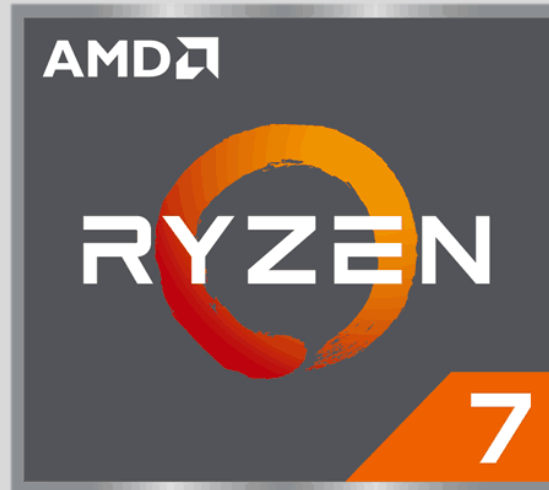
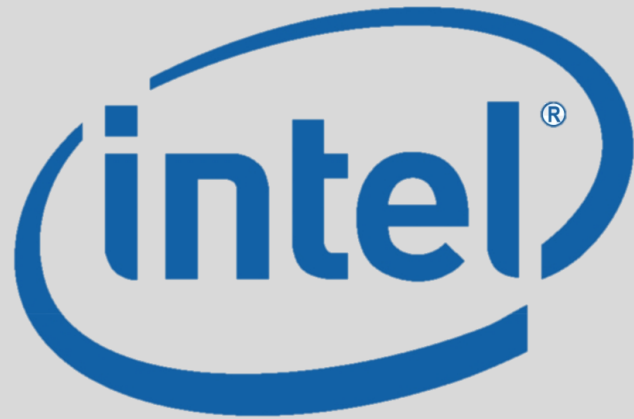




CENTRAL PROCESSING UNIT (CPU)

- Often called the “brain” of the computer
- Responsible for executing instructions that make software and operating system work.
- Task:
 - Process data
 - Perform calculations (math and logic)
 - Control another component of the computer
 - Executes instructions one by one (or few at a time)
 - Great for general-purpose thinking

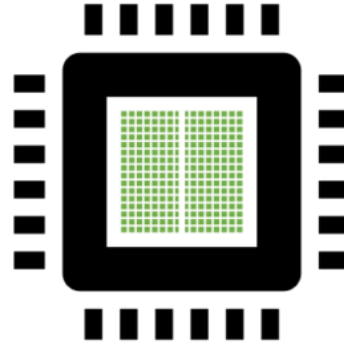
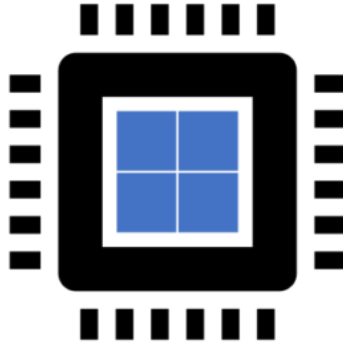
CENTRAL PROCESSING UNIT (CPU)



GRAPHICS PROCESSING UNIT (GPU)



- Is a specialized processor designed to handle graphics and visual data
- Good at doing many calculations at the same time → ideal for images, videos and complex computation
- The muscle:
 - Does heavy lifting
 - Execute massive numbers of simple operations in parallel
 - Ideal for repetitive, compute-intensive tasks:
 - Rendering millions of pixels
 - Matrix calculation
 - Image, video, and AI workload
- The visual cortex:
 - Processes vast amounts of visual data simultaneously
 - Detects patterns, edges, motion, and depth



CPU	GPU
Central Processing Unit	Graphics Processing Unit
4-8 Cores	100s or 1000s of Cores
Low Latency	High Throughput
Good for Serial Processing	Good for Parallel Processing
Quickly Process Tasks That Require Interactivity	Breaks Jobs Into Separate Tasks To Process Simultaneously
Traditional Programming Are Written For CPU Sequential Execution	Requires Additional Software To Convert CPU Functions to GPU Functions for Parallel Execution

GPU VS CPU

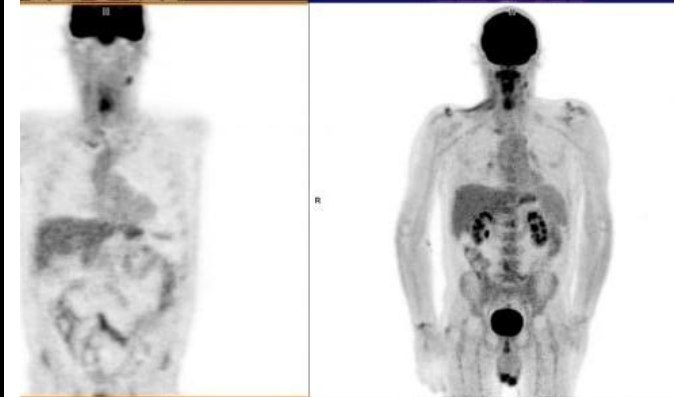
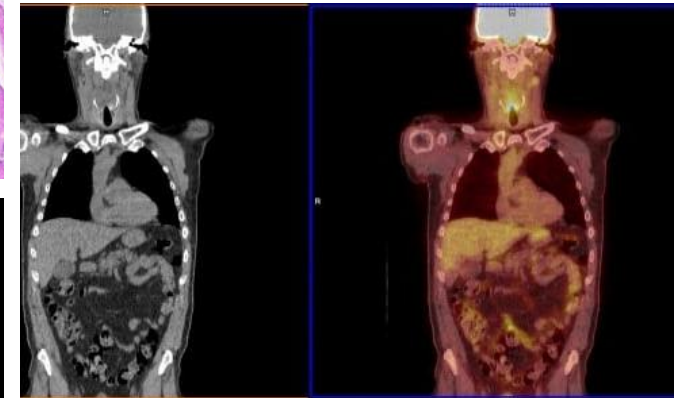
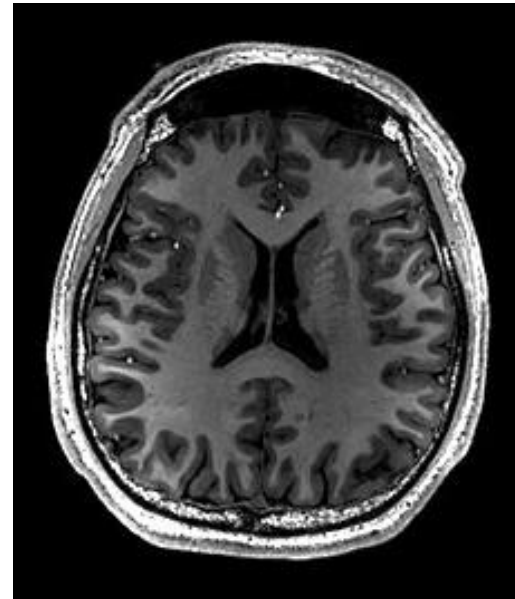
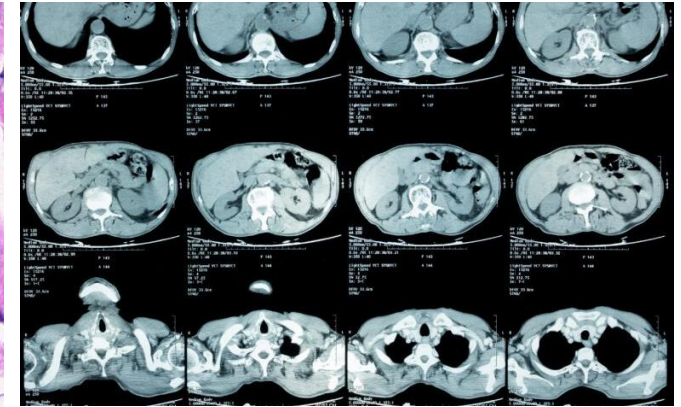
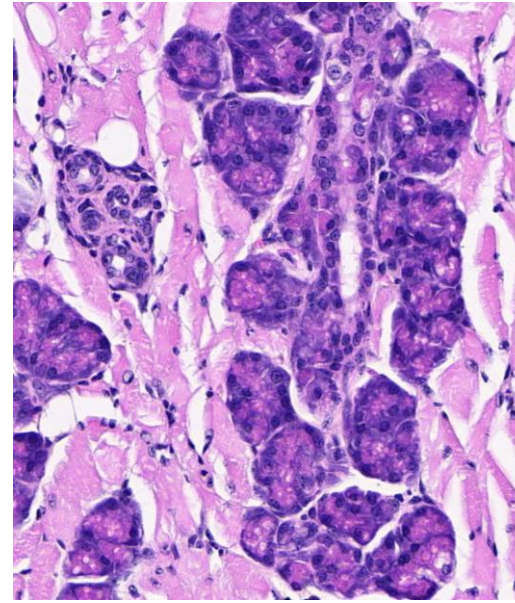


WHY GPU IS IMPORTANT FOR MEDICAL IMAGE ANALYSIS?



BIG METRICS DATA

- Each image may contain millions of pixels/voxels,
 - CT: hundreds to thousands of slices per scan
 - MRI: 3D + multi-contrast + time (4D data)
 - Ultrasound: real-time image sequences
 - PET/SPECT: volumetric + time + noise modeling
- Most medical image analysis operations apply the same computation to every pixels/voxels
- Why this matters? ->
 - CPU: few cores → slow for pixel-wise operations
 - GPU: thousand of cores → massive speed -up
- GPU can process all pixels/voxels simultaneously, while CPUs would process them mostly sequentially



HIGH-RESOLUTION SCANNING

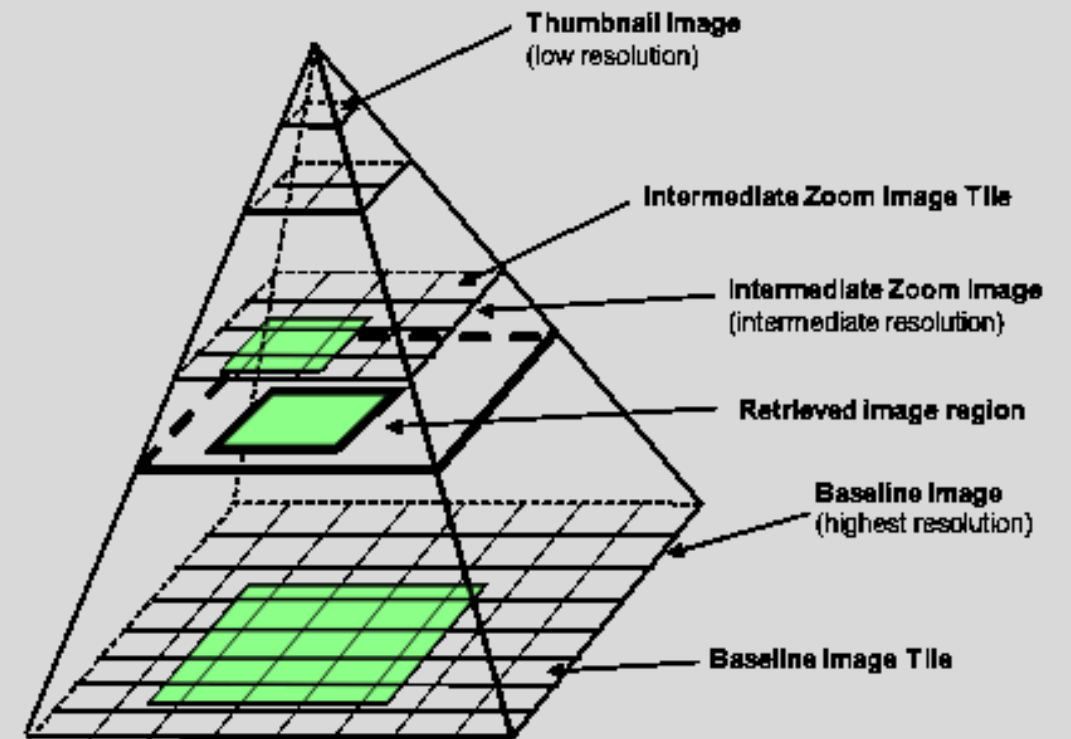
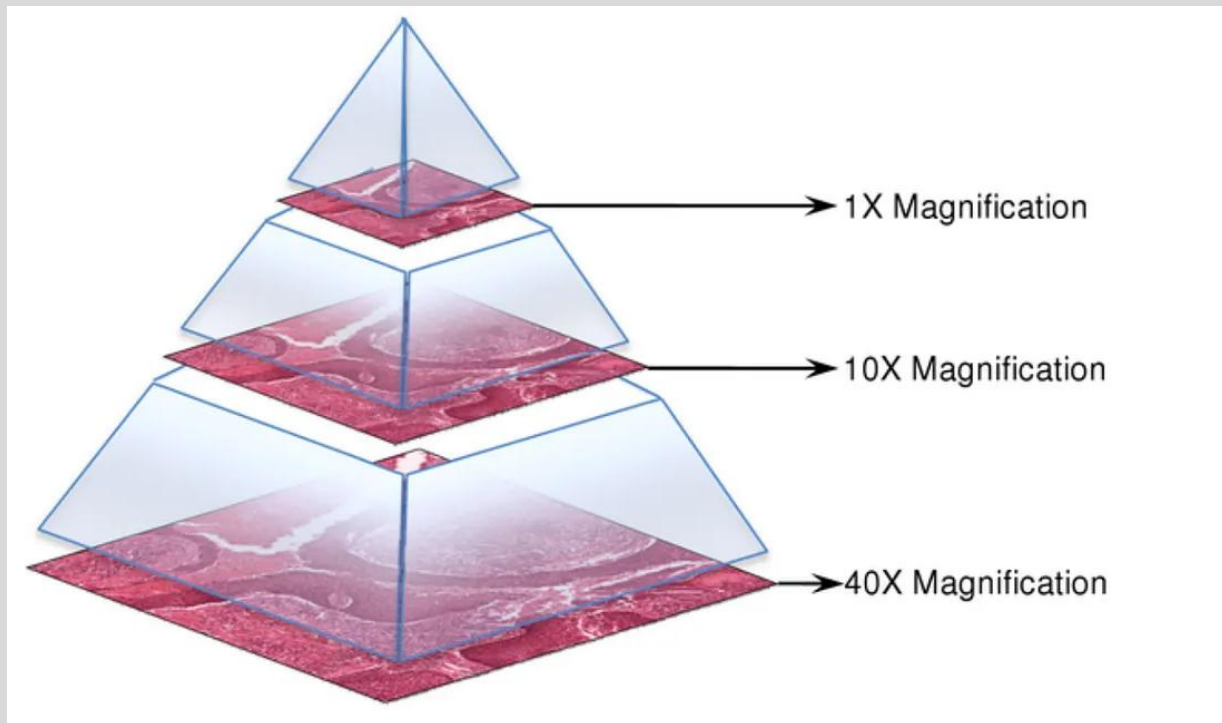
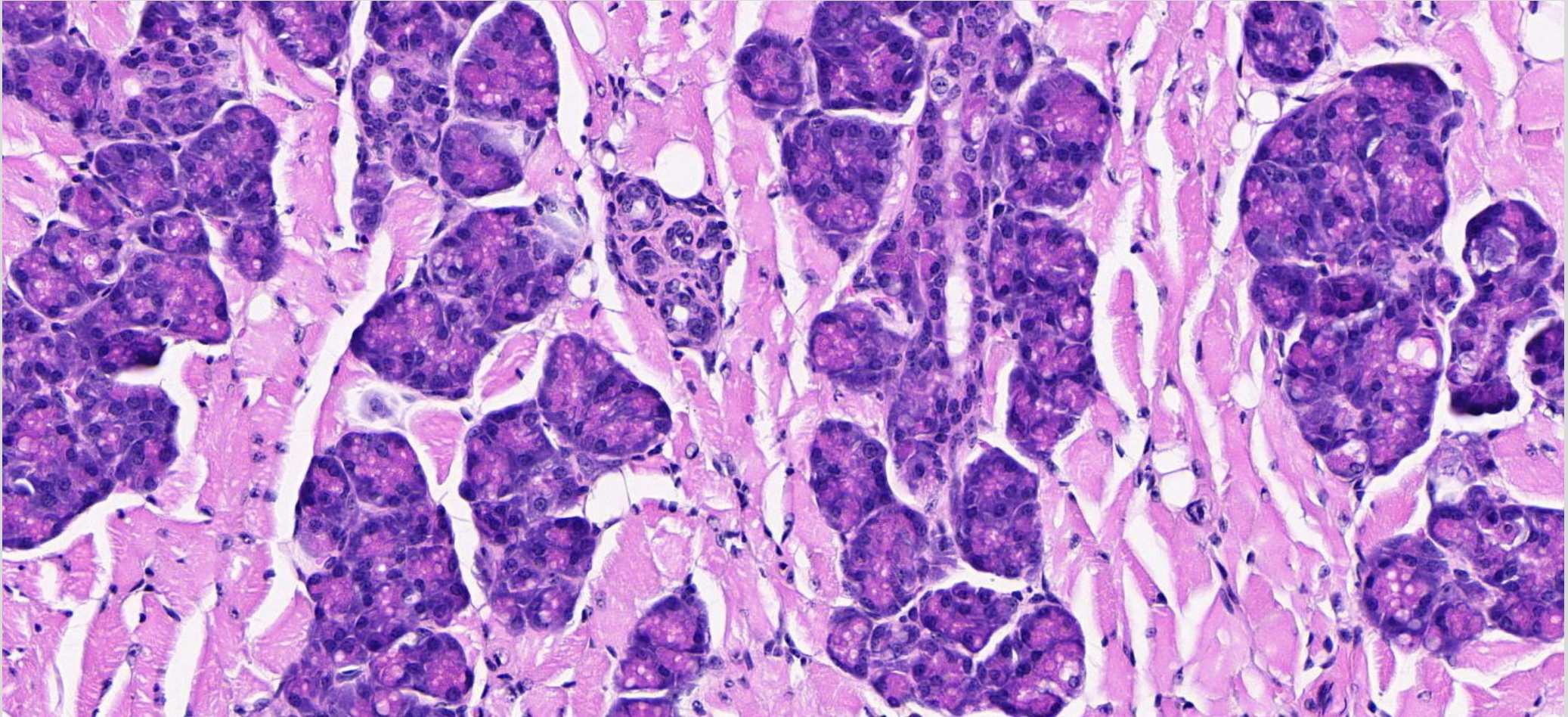


Illustration of how digital slides are stored in pyramid structure on Whole Slide Images (WSI)

HIGH-RESOLUTION SCANNING



Demo WSI: <https://demo.ihisto.io/WebViewer>

3

TYPES OF GPU AND THEIR ADVANTAGES

NVIDIA



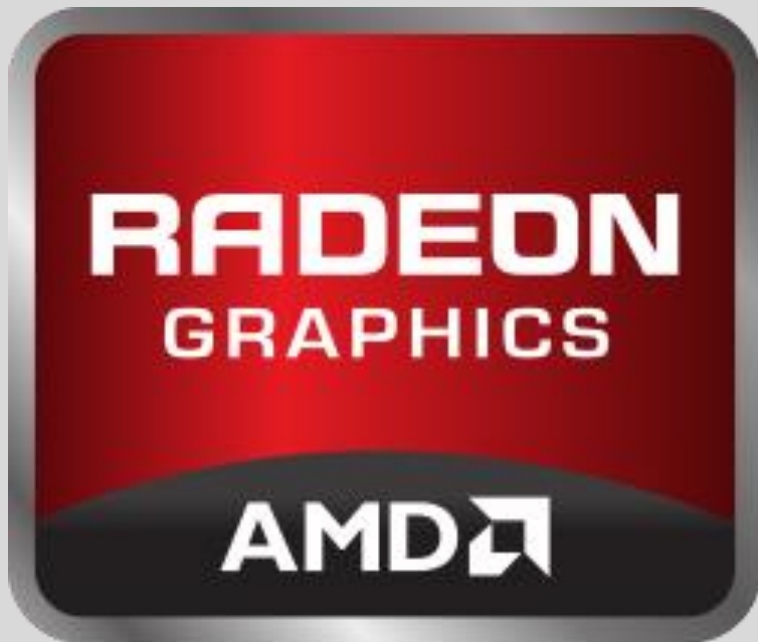
NVIDIA®

- Use CUDA (Computer Unified Device Architecture) architecture.
- Almost all PCM libraries (PyTorch, TensorFlow, OpenCV) are optimized for NVIDIA.
- NVIDIA GPU Types:
 - NVIDIA GeForce (GTX/RTX)
 - NVIDIA RTX/Quadro (Workstation GPU)
 - NVIDIA Data Center GPU (Tesla/A-Series/H-Series)
 - NVIDIA Jetson
 - NVIDIA DRIVE & Clara (Industry Specific)

NVIDIA

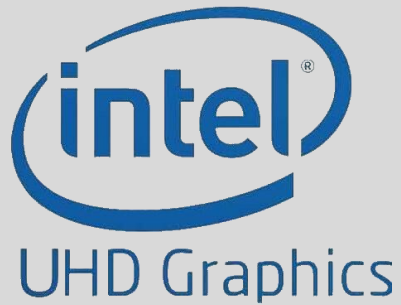
NVIDIA Series	Main Target	Main Advantages
GeForce (GTX/RTX)	Consumers & research	Cheap, flexible, fast
RTX / Quadro	Profesional	Stable, precision, medical
Data Center (A/H)	Huge AI	Big VRAM, super fast
Jetson	Embedded	Real-time, low power
Clara	Medical	AI medical pipeline

AMD (RADEON)



- AMD Radeon is a line from AMD (Advanced Micro Devices) → equivalent to NVIDIA GeForce/RTX.
- While NVIDIA is strong in the CUDA ecosystem, AMD Radeon excels in its open architecture and price-performance ratio.
- Types of AMD Radeon:
 - Radeon RX → consumer & gaming
 - Radeon Pro → professional & workstation
 - AMD Instinct → data center & large-scale AI

INTEGRATED GPU (INTEL IRIS / APPLE M-SERIES)



- Integrated GPU (iGPU): A GPU integrated with the CPU, not a separate device.
- Key features:
 - Shares memory with system RAM
 - Low power consumption
 - Compact size
 - Sufficient performance for light to medium tasks
- iGPUs from Intel:
 - Intel UHD Graphics → entry level
 - Intel Iris Plus/Iris Xe □ more powerful generation
- Apple M-Series

INTEGRATED GPU (INTEL IRIS/APPLE M-SERIES)

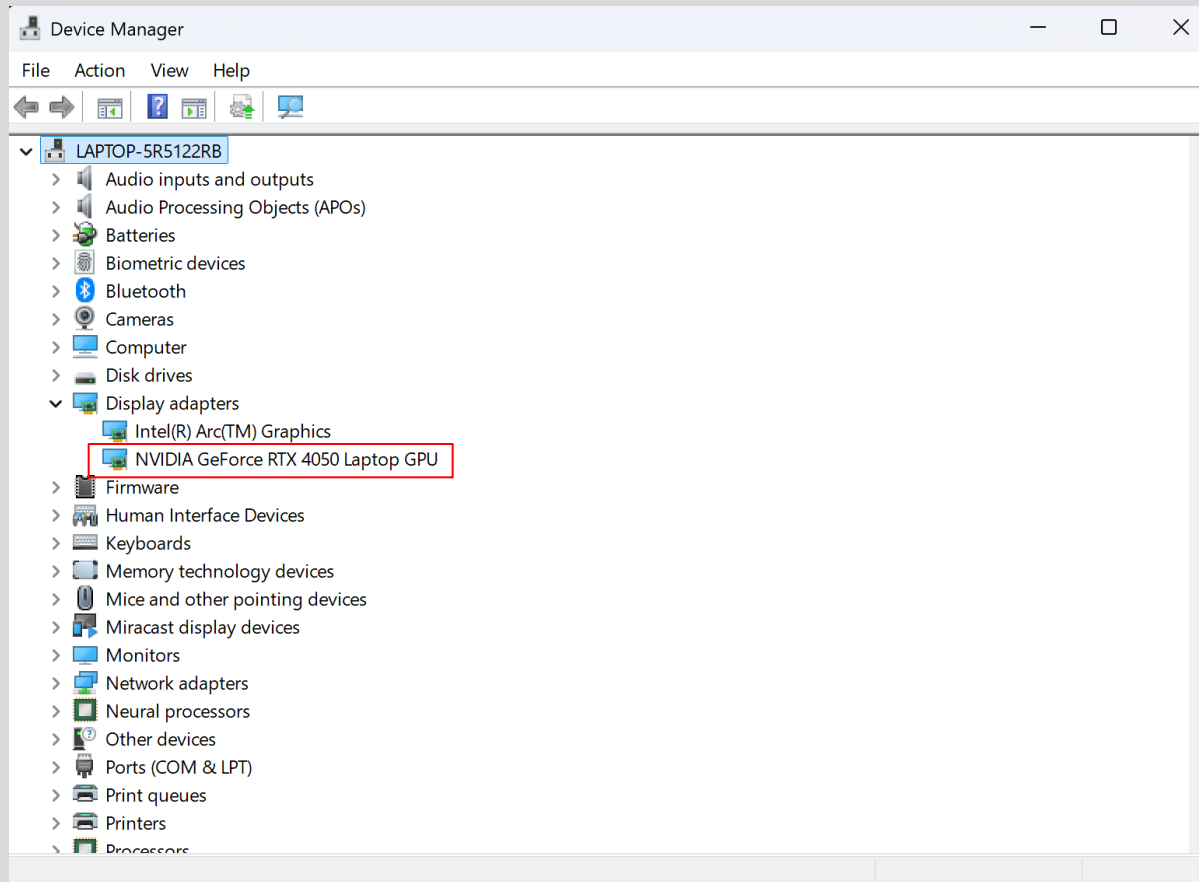
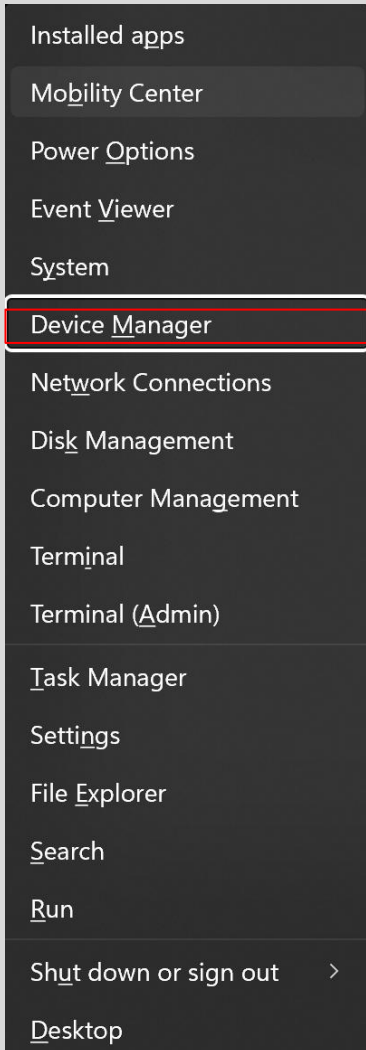
Aspect	Intel Iris	Apple M-Series
Type	Integrated GPU	Integrated SoC GPU
Memory	Shared RAM	Unified Memory
Power	Low	Very efficient
AI support	Limited	Neural Engine
Image processing	✓ Good	✓ ✓ Very Good
DL training	✗	✗
DL inference	⚠	✓



GPU SETUP

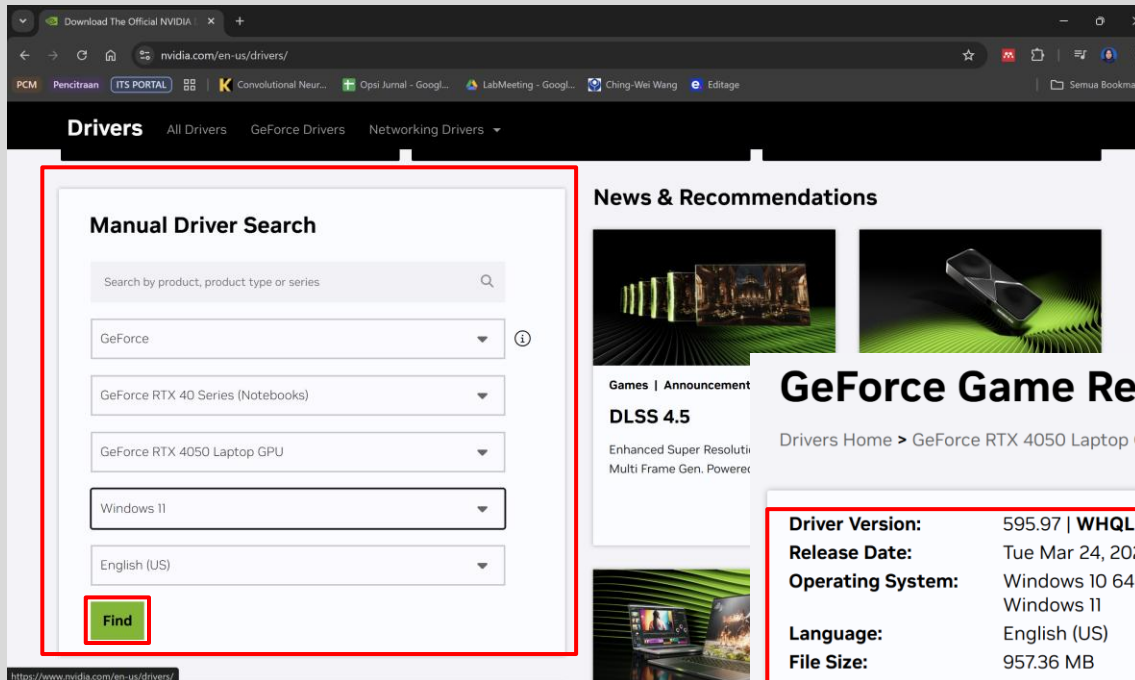


0. HARDWARE CHECK



1. Press Win+X → Device Manager
2. Go to Display adapters
3. Make sure there is: NVIDIA GTX/RTX

1. INSTALL NVIDIA (1)



1. Download the official driver:
 1. Open:
<https://www.nvidia.com/Download/index.aspx>
 2. Choose:
 1. Product type: GeForce
 2. Product series: according to the available GPU types (RTX30, RTX40, etc.)
 3. OS: Windows 10/11
 4. Choose the Game Ready Driver to get the saver option for CUDA

GeForce Game Ready Driver 595.97 | Windows 11

Drivers Home > GeForce RTX 4050 Laptop GPU | Windows 11 > GeForce Game Ready Driver

Driver Version: 595.97 | WHQL
Release Date: Tue Mar 24, 2026
Operating System: Windows 10 64-bit, Windows 11
Language: English (US)
File Size: 957.36 MB

*This download includes the NVIDIA graphics driver and an option to additionally install the NVIDIA App. Details for use of the software can be found in the [NVIDIA GeForce Software license](#) and [NVIDIA App License Agreement](#) respectively.

[Download](#)

Get Automatic Driver Updates

The NVIDIA App is the essential companion for PC gamers and creators. Keep your PC up to date with the latest NVIDIA drivers and technology.

[NVIDIA App](#)

Get The Latest NVIDIA GeForce Updates.

Sign up for gaming and entertainment deals, announcements, and more from NVIDIA.

[NVIDIA Privacy Policy](#)

[Submit](#)

1. INSTALL NVIDIA (2)



2. Install the drivers:
 1. Run the drivers
 2. Choose Custom (Advanced)
 3. Checklist Clean install
 4. Restart
3. Check if the installation succeed:
 1. Open Command Prompt
 2. Type: `nvidia-smi`:
 - Succeed: GPU table appears,
 - Failed: error/ GPU table not available

```
C:\Users\ >nvidia-smi
Mon Apr 6 13:08:07 2026

+-----+
| NVIDIA-SMI 546.67 | Driver Version: 546.67 | CUDA Version: 12.3 |
+-----+
| GPU  Name          | TCC/WDDM | Bus-Id | Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf    | Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                      |            |          |         |      GPU-MIG  Compute M. |
+-----+-----+
| 0  NVIDIA GeForce RTX 4050 ... WDDM | 00000000:01:00.0 Off |          0%      Default |
| N/A   44C    P3      | 8W / 45W    | 0MiB / 6141MiB |          0%      Default |
+-----+-----+

+-----+
| Processes: |
| GPU  GI  CI | PID  Type  Process name                      | GPU Memory |
| ID        ID |      |          | Usage |
+-----+-----+
| No running processes found |
+-----+
```

2. INSTALL VS CODE

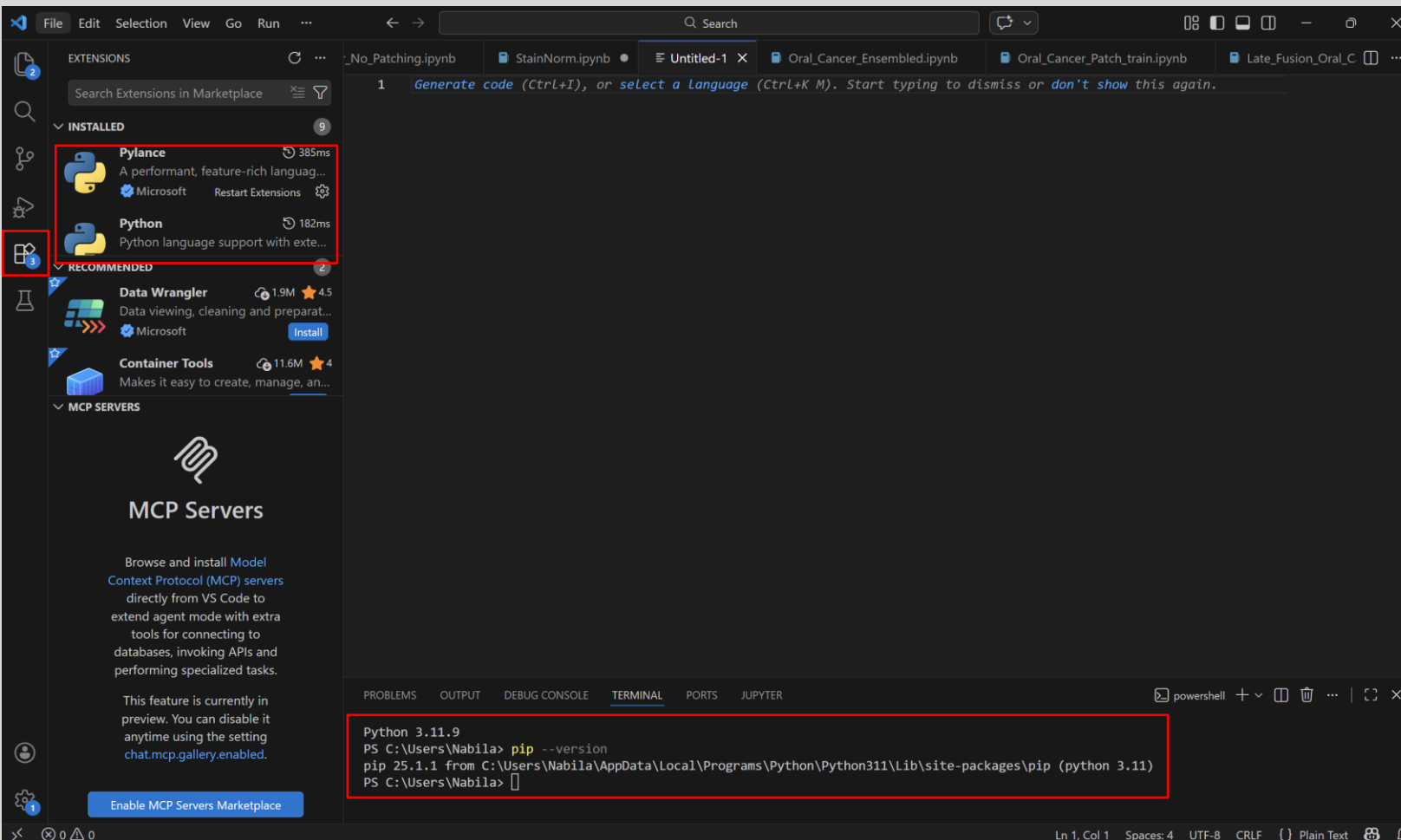
1. Download vs code from:
<https://code.visualstudio.com/>
2. Required extensions:
 1. Python
 2. Pylance
 3. (optional) Jupyter

3. Install python:
<https://www.python.org/downloads/>

4. While installing, check **Add Python to PATH**

- Verification:

```
1 python --version
2 pip --version
```



3. CREATE VIRTUAL ENV.

1. Create Project folder

```
PS C:\Users\Nabila> cd ~
PS C:\Users\Nabila> cd ..
PS C:\Users> cd ..
PS C:\> cd "C:\Users\Nabila\Documents\RISET\Breast_Cancer_IHC\"
PS C:\Users\Nabila\Documents\RISET\Breast_Cancer_IHC> mkdir BC-IHC
```

Directory: C:\Users\Nabila\Documents\RISET\Breast_Cancer_IHC

Mode	LastWriteTime	Length	Name
d----	4/7/2026 10:29 AM		BC-IHC

2. Create virtual environment

```
PS C:\Users\Nabila\Documents\RISET\Breast_Cancer_IHC> cd BC-IHC
PS C:\Users\Nabila\Documents\RISET\Breast_Cancer_IHC\BC-IHC> python -m venv bc_ihc
```

3. Activate the virtual environment

```
PS C:\Users\Nabila\Documents\RISET\Breast_Cancer_IHC\BC-IHC> bc_ihc\Scripts\activate
(bc_ihc) PS C:\Users\Nabila\Documents\RISET\Breast_Cancer_IHC\BC-IHC> 
```

4. INSTALL CUDA TOOLKIT (1)

- CUDA ≠ Driver
- CUDA can be installed using 4 different ways:
 - Official CUDA Toolkit (classic)
 - CUDA via Conda
 - CUDA via PyTorch / Pip (CUDA runtime only)
 - CUDA via Docker
 - (Additional) CUDA via WSL (Linux on Windows)
- Note: Not every user need a full version of CUDA Toolkit

4. INSTALL CUDA TOOLKIT (2)

1. Official CUDA Toolkit (Classic)

- Install from the NVIDIA website, includes:
 - Nvcc
 - CUDA headers
 - CUDA runtime
- Useful if:
 - Writing CUDA C/ C++
 - Compiling your own kernel
 - Low-level GPU research
- Suitable for:
 - CUDA developers
 - Low-level GPU computing research

1. Check the recommendation of CUDA version:
PyTorch: CUDA 11.8 or 12.x
Tensorflow: CUDA 11.8
 2. Download the CUDA:
 1. Open: <https://developer.nvidia.com/cuda-downloads>
 2. Choose:
OS: Windows
Architecture: x86_64
Version: 11
Installer: exe (local)
 3. Install
 4. Restart
3. Check if the installation succeed:
 1. Open Command Prompt
 2. Type: `nvcc --version:`
Succed: type of CUDA appears,
Failed: command unknown

4. INSTALL CUDA TOOLKIT (3)

2. CUDA via Conda

- Concept:
 - CUDA installed in virtual environment
 - Not disturbing the system
 - Do not need to install global CUDA Toolkit
- Advantages:
 - No need to install CUDA Toolkit manually
 - Guaranteed compatibility with CUDA version
 - Save for multiple environments
- Very suitable for:
 - Practical work
 - Research
 - Sharing environments

```
1 conda create -n torch-gpu python=3.10
2 conda activate torch-gpu
3 conda install pytorch torchvision torchaudio pytorch-cuda=11.8
  c pytorch -c nvidia
```

4. INSTALL CUDA TOOLKIT (4)

3. CUDA via Pip / PyTorch Wheels (Runtime Only)

- Concept:
 - CUDA RUNTIME is installed
 - No full CUDA Toolkit
 - No nvcc
- Advantages:
 - Fastest setup
 - No need to manually install CUDA
 - Compatible by design
- Very suitable for:
 - Deep learning
 - Image processing
 - MIA
 - Jupyter/VS Code workflow

```
1 pip install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cu118
```

4. INSTALL CUDA TOOLKIT (5)

4. CUDA via Docker (Professional/Server)

- Concept:
 - CUDA is inside the container
 - The host OS only needs:
 - NVIDIA driver
 - NVIDIA Container Toolkit
- Advantages:
 - No dependency conflicts
 - 100% consistent environment
 - Many hospitals and labs use this
- Very suitable for:
 - Lab server
 - Multi-user projects
 - Medical deployments
 - Research reproducibility

```
1 nvidia/cuda:11.8.0-cudnn8-runtime-ubuntu22.04
```

4. INSTALL CUDA TOOLKIT (6)

4. CUDA via WSL (Linux in Windows)

- Concept:
 - CUDA runs on Linux (WSL2)
 - NVIDIA driver on Windows
 - CUDA Toolkit on Linux
- Advantages:
 - More stable for AI
 - Server-like environment
- Very suitable for:
 - Linux developers
 - AI research
 - Server-like environment on a laptop

5. CUDA VERIFICATION IN PYTORCH

- Create python file:

```
import torch
print(torch.__version__)
print("CUDA available:", torch.cuda.is_available())
print("Device:", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU only")
```

- The output should be:

```
2.7.1+cu118
CUDA available: True
Device: NVIDIA GeForce RTX 4050 Laptop GPU
```


UJI COBA

```
Epoch [99] - Loss: 0.0196
Epoch [100] - Loss: 0.0166
```

Training stopped at epoch 100 (loss < 0.001)

Confusion Matrix:

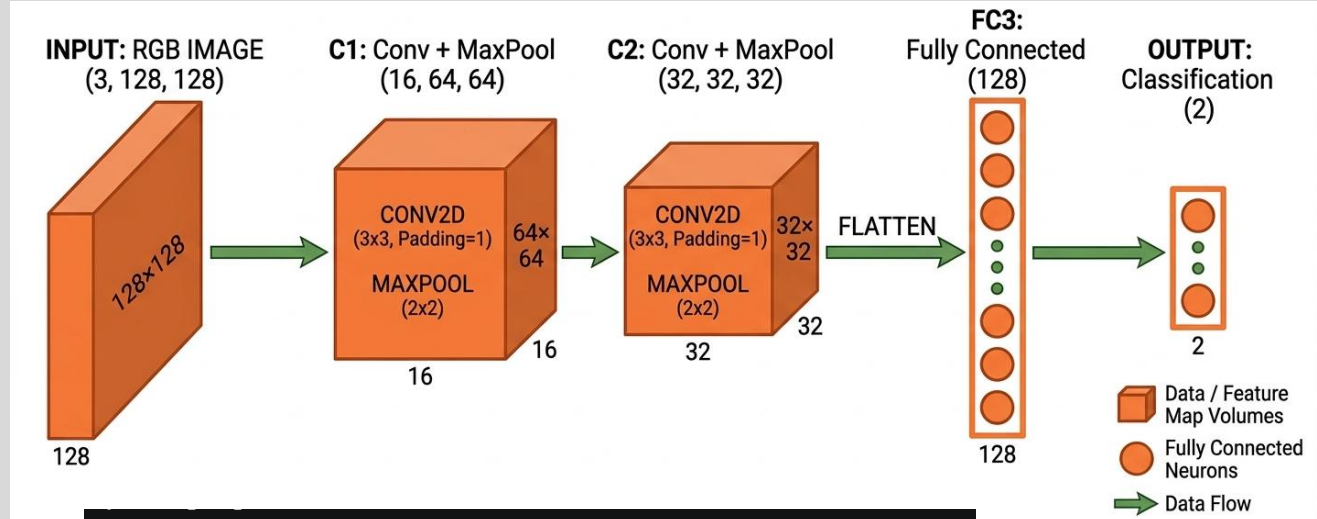
```
[[ 7 11]
 [ 4 84]]
```

Classification Report:

	precision	recall	f1-score	support
Normal	0.64	0.39	0.48	18
OSCC	0.88	0.95	0.92	88
accuracy			0.86	106
macro avg	0.76	0.67	0.70	106
weighted avg	0.84	0.86	0.84	106

Total waktu training pada cuda: 2695.14 detik
Rata-rata waktu per epoch: 26.95 detik

GPU



```
Epoch [99] - Loss: 0.0504
Epoch [100] - Loss: 0.0630
```

Training stopped at epoch 100 (loss < 0.001)

Confusion Matrix:

```
[[ 6 12]
 [ 4 84]]
```

Classification Report:

	precision	recall	f1-score	support
Normal	0.60	0.33	0.43	18
OSCC	0.88	0.95	0.91	88
accuracy			0.85	106
macro avg	0.74	0.64	0.67	106
weighted avg	0.83	0.85	0.83	106

Total waktu training pada cpu: 2977.83 detik
Rata-rata waktu per epoch: 29.78 detik

CPU

OPSI LAIN

Kaggle



1. Buka Kaggle notebook
2. Pengaturan
3. Pilih accelerator → GPU
4. Cek : !nvidia-smi
5. Cek Pytorch CUDA

<https://www.kaggle.com/code/nabilata/gpu-activation-cuda/edit>

Google Colab



1. Buka Colab notebook
2. Buka Runtime → Change runtime type
3. Cek : !nvidia-smi
4. Cek Pytorch CUDA

https://colab.research.google.com/drive/1FQnU_ZMwS7ZOs-BVcXS6Er_eJhIP9T74?authuser=1#scrollTo=rnGXUm24rRJM

Aspek	Local GPU (VS Code)	Kaggle GPU	Google Colab GPU
Kepemilikan GPU	✓ Milik sendiri	✗ Milik cloud Kaggle	✗ Milik Google
Jenis GPU	Bebas (GTX / RTX / A-series)	Tesla T4 / P100 / (kadang A100)	Tesla T4 / P100 / V100 / A100*
Kontrol GPU	✓ Penuh	✗ Tidak bisa memilih	✗ Tidak bisa memilih
CUDA setup	Manual / pip / conda	Otomatis	Otomatis
Environment	✓ Persisten	✗ Temporary	✗ Temporary
Virtual environment	✓ Ya	✗ Tidak	✗ Tidak
Instalasi library	✓ Permanen	✗ Hilang saat reset	✗ Hilang saat reset
Akses filesystem	✓ Penuh	⚠ Terbatas	⚠ Terbatas
Internet akses	✓ Bebas	⚠ Terbatas (aturan Kaggle)	✓ Bebas
Multi-GPU	✓ Bisa	✗ Tidak	✗ Tidak
Job background / lama	✓ Bisa	✗ Tidak	✗ Tidak
Cocok untuk riset besar	✓ ✓ ✓	⚠	⚠
Cocok untuk praktikum	⚠ Setup berat	✓ ✓	✓ ✓ ✓

PERBANDINGAN

Platform	Limit Running Time
Local GPU (VS Code)	✅ Tidak terbatas (selama listrik & hardware ada)
Kaggle GPU	🕒 ± 9–12 jam per session 📊 ± 30 jam GPU per minggu
Google Colab (Free)	🕒 ± 12 jam per session 😴 Idle timeout ± 60–90 menit
Google Colab Pro / Pro+	🕒 Lebih lama & lebih stabil (tidak dijamin unlimited)

LIMIT RUNNING TIME

THANK YOU